

PATH TRACER (SDN-BASED)

Computer Networks (UE24CS252B)

Name: Ankit Bembalgi

SRN: PES2UG24AM020

1. Problem Statement

This project implements a **Software Defined Networking (SDN)-based Path Tracing Tool** to:

- Identify and display the path taken by packets between hosts
- Track flow rules installed in switches
- Identify forwarding paths dynamically
- Display route information clearly
- Validate correctness using test cases

The system leverages SDN principles to provide centralized control and improved visibility into network behavior.

2. SDN Architecture

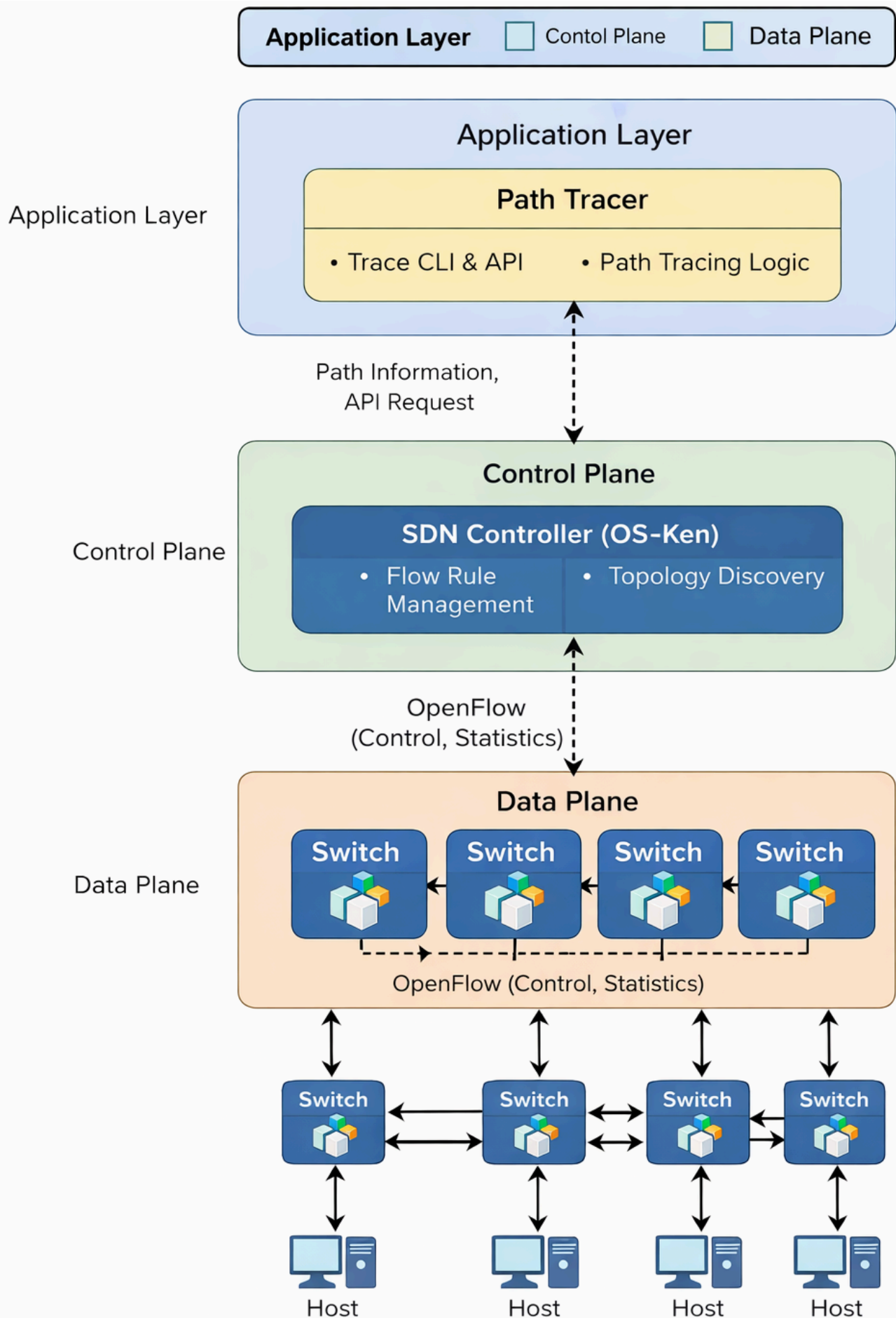
Software Defined Networking (SDN) separates the control plane from the data plane, enabling centralized management of network behavior.

Key Components:

- **Controller (OS-Ken):**
Acts as the brain of the network, managing flow rules and decision-making.
- **Data Plane (Switches):**
OpenFlow-enabled switches that forward packets based on rules.
- **Application Layer (Path Tracer):**
Implements path tracing logic and exposes APIs for retrieving route information.

Features of the Architecture:

- Centralized control using OS-Ken (Ryu fork)
- OpenFlow 1.3 protocol for communication
- Dynamic topology discovery
- Reactive flow installation



SDN-based Path Tracing Architecture
Architecture Diagram

3. Mininet Topology Design

The project uses a custom Mininet topology consisting of:

- **4 Hosts:** h1, h2, h3, h4
- **4 Switches:** s1, s2, s3, s4

Topology Characteristics:

- Mesh-like structure (not linear or tree)
- Multiple interconnections:
 - s1-s2
 - s1-s3
 - s2-s3
 - s2-s4
 - s3-s4

This design allows multiple possible paths, enabling shortest-path computation.

Configuration Details:

- Default Mininet link parameters
- No bandwidth or delay constraints
- IPv6 disabled for stability
- NIC offloading disabled on hosts

```
(.venv) [shadow_e15@Shadow] (~ /College/CN/SDN_Mini_Project) - [git://master ✓] -  
L> sudo python sdn_path_tracer/demo_topology.py --controller-ip 127.0.0.1 --controller-port 6653  
  
*** Creating network  
*** Adding hosts:  
h1 h2 h3 h4  
*** Adding switches:  
s1 s2 s3 s4  
*** Adding links:  
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s1, s3) (s2, s3) (s2, s4) (s3, s4)  
*** Configuring hosts  
h1 h2 h3 h4  
*** Starting controller  
c0  
*** Starting 4 switches  
s1 s2 s3 s4 ...  
Topology started. Try: h1 ping -c 2 h4  
*** Starting CLI:  
mininet>
```

```
mininet> net  
h1 h1-eth0:s1-eth1  
h2 h2-eth0:s2-eth1  
h3 h3-eth0:s3-eth1  
h4 h4-eth0:s4-eth1  
s1 lo: s1-eth1:h1-eth0 s1-eth2:s2-eth2 s1-eth3:s3-eth2  
s2 lo: s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s4-eth2 s2-eth4:s3-eth4  
s3 lo: s3-eth1:h3-eth0 s3-eth2:s1-eth3 s3-eth3:s4-eth3 s3-eth4:s2-eth4  
s4 lo: s4-eth1:h4-eth0 s4-eth2:s2-eth3 s4-eth3:s3-eth3  
c0  
mininet>
```

4. Controller Implementation

The controller is implemented using **OS-Ken** and resides in:

sdn_path_tracer/controller.py

Core Responsibilities:

- Handles `packet_in` events
- Learns host locations dynamically
- Discovers network topology
- Computes shortest path between switches
- Installs flow rules along the path
- Installs reverse path rules
- Maintains flow tracking data
- Exposes a trace API

Algorithm Used:

- **Breadth-First Search (BFS)**
- Computes shortest path based on hop count
- Treats all links with equal weight

```
(.venv) [shadow_e15@Shadow] - (~ /College/CN/SDN_Mini_Project) - [git://master x] -  
L> source .venv/bin/activate  
python -m sdn_path_tracer.run_controller --observe-links --ofp-tcp-listen-port 6653  
--trace-api-port 8080  
  
2026-04-17 12:25:33,330 INFO os_ken.base.app_manager: loading app sdn_path_tracer.co  
ntroller  
2026-04-17 12:25:33,330 INFO os_ken.base.app_manager: loading app os_ken.topology.sw  
itches  
2026-04-17 12:25:33,331 INFO os_ken.base.app_manager: loading app os_ken.controller.  
ofp_handler  
2026-04-17 12:25:33,331 INFO os_ken.base.app_manager: instantiating app sdn_path_tra  
cer.controller of PathTracerController  
2026-04-17 12:25:33,331 INFO os_ken.base.app_manager: instantiating app os_ken.topol  
ogy.switches of Switches  
2026-04-17 12:25:33,332 INFO os_ken.base.app_manager: instantiating app os_ken.contr  
oller.ofp_handler of OFPHandler  
2026-04-17 12:25:33,335 INFO PathTracerController: Trace API listening on http://127  
.0.0.1:8080  
2026-04-17 12:42:50,154 INFO PathTracerController: trace-api "GET /trace?src=h1&dst=  
h4 HTTP/1.1" 200 -  
2026-04-17 12:55:35,476 WARNING os_ken.topology.switches: Multiple connections from  
000000000000000001  
2026-04-17 12:55:35,477 WARNING os_ken.topology.switches: Multiple connections from  
000000000000000003  
2026-04-17 12:55:35,478 WARNING os_ken.topology.switches: Multiple connections from  
000000000000000004  
2026-04-17 12:55:35,479 WARNING os_ken.topology.switches: Multiple connections from  
000000000000000002
```

5. Flow Rule Management

The system uses **reactive SDN**:

- Flow rules are installed **on-demand**
- Triggered by incoming packets

Matching Fields Used:

- `in_port`
- `eth_src`
- `eth_dst`

Characteristics:

- MAC-based forwarding
- Dynamic rule installation
- Bidirectional flow setup
- Efficient rule tracking for tracing

```
(.venv) [shadow_e15@Shadow] (~/College/CN/SDN_Mini_Project)-[git://master ✓]-
L> sudo python sdn_path_tracer/demo_topology.py --controller-ip 127.0.0.1 --controller-port 6653

*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s1, s3) (s2, s3) (s2, s4) (s3, s4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
Topology started. Try: h1 ping -c 2 h4
*** Starting CLI:
mininet> █
```

6. Functionality Implementation

The project provides the following functionality:

1. Path Tracing

Displays the full path:

h1 -> s1 -> s2 -> s4 -> h4

2. CLI Interaction

- Mininet CLI for traffic generation
- Custom trace CLI for querying paths

3. Logging

- Controller logs network activity
- Trace API requests logged

4. Flow Visualization

- Text-based visualization of:
 - Switch hops
 - Flow rules
 - Forwarding path

5. API Support

- HTTP-based trace API
 - JSON output support
-

```
(.venv) [shadow_e15@Shadow] (~/College/CN/SDN_Mini_Project) -[git://master x]-  
L> python -m sdn_path_tracer.trace_cli --src h1 --dst h4  
  
Packet path:  
h1 -> s1 -> s2 -> s4 -> h4  
Switch hops:  
s1, s2, s4  
Flow rules:  
s1: in_port=1 out_port=2 match={'in_port': 1, 'eth_src': '00:00:00:00:00:01', 'eth_  
dst': '00:00:00:00:00:04'}  
s2: in_port=2 out_port=3 match={'in_port': 2, 'eth_src': '00:00:00:00:00:01', 'eth_  
dst': '00:00:00:00:00:04'}  
s4: in_port=2 out_port=1 match={'in_port': 2, 'eth_src': '00:00:00:00:00:01', 'eth_  
dst': '00:00:00:00:00:04'}
```

7. Performance Evaluation

The system was evaluated using connectivity and latency tests.

Testing Method:

- ICMP ping between hosts
- Validation of packet delivery
- Observation of latency

Results:

- 0% packet loss
- Stable connectivity
- Low latency after initial packet

```
mininet> h1 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=13.4 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.452 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.137 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=0.156 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=0.111 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4081ms
rtt min/avg/max/mdev = 0.111/2.844/13.368/5.263 ms
mininet> exit
*** Stopping 1 controllers
```

8. Python Script (Key Snippets)

Controller Initialization

```
class PathTracerController(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
```

BFS Path Computation

```
def shortest_path(self, src, dst):
    visited = set()
    queue = [(src, [src])]
```

Flow Installation

```
match = parser.OFPMatch(
    in_port=in_port,
    eth_src=src_mac,
    eth_dst=dst_mac
)
```

```
def shortest_path(self, src_dpid: int, dst_dpid: int) -> list[int]:
    if src_dpid == dst_dpid:
        return [src_dpid]
    queue = deque([(src_dpid, [src_dpid])])
    visited = {src_dpid}
    while queue:
        current, path = queue.popleft()
        for neighbor in sorted(self.adjacency.get(current, {})):
            if neighbor in visited:
                continue
            next_path = path + [neighbor]
            if neighbor == dst_dpid:
                return next_path
            visited.add(neighbor)
            queue.append((neighbor, next_path))
    raise TraceError(f"No path exists between s{src_dpid} and s{dst_dpid}.")
```

9. Screenshots of Demo

```
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s1, s3) (s2, s3) (s2, s4) (s3, s4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
Topology started. Try: h1 ping -c 2 h4
*** Starting CLI:
mininet> h1 ping -c 5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=10.4 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.596 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.139 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.132 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.160 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4077ms
rtt min/avg/max/mdev = 0.132/2.276/10.353/4.042 ms
mininet> █
```

```
2026-04-17 13:12:04,391 WARNING os_ken.topology.switches: Multiple connections from
0000000000000001
2026-04-17 13:12:04,393 WARNING os_ken.topology.switches: Multiple connections from
0000000000000004
2026-04-17 13:12:04,393 WARNING os_ken.topology.switches: Multiple connections from
0000000000000002
2026-04-17 13:12:04,395 WARNING os_ken.topology.switches: Multiple connections from
0000000000000003
```

```
[shadow_e15@Shadow] - (~ /College/CN/SDN_Mini_Project) - [git://master ✓] -
> python -m sdn_path_tracer.trace_cli --src h1 --dst h3

Packet path:
  h1 -> s1 -> s3 -> h3
Switch hops:
  s1, s3
Flow rules:
  s1: in_port=1 out_port=3 match={'in_port': 1, 'eth_src': '00:00:00:00:00:01', 'eth
dst': '00:00:00:00:00:03'}
  s3: in_port=2 out_port=1 match={'in_port': 2, 'eth_src': '00:00:00:00:00:01', 'eth
dst': '00:00:00:00:00:03'}
```

10. GitHub Repository Link

Project Repository:

https://github.com/AnkitB1405/SDN_Mini_Project

11. Conclusion

The SDN-based Path Tracing Tool successfully demonstrates how centralized control can enhance network visibility and management. By leveraging OS-Ken and OpenFlow, the system dynamically installs flow rules and reconstructs packet paths in real time.

The use of BFS ensures efficient shortest-path routing, while the CLI and API interfaces provide accessible and clear insights into network behavior.

This project highlights the advantages of SDN in debugging, monitoring, and understanding packet forwarding, making it a valuable tool for network analysis.